Universidad de El Salvador · Facultad Multidisciplinaria Oriental

Departamento de Ingeniería y Arquitectura · Especialización en BD e Inteligencia de Negocios

Manual Breve de Instalación y Ejecución

Sistema Operacional (OLTP), Data Warehouse (OLAP), Proceso ETL y Plataforma BI (Streamlit)

Caso de Estudio: Empresa Comercializadora Centro de Plásticos El Limeño

Equipo de Desarrollo: Cristian Pineda (PB20002) · Andrés Vásquez (VV18009) · Luis Parada (PB19022)
Docente Asesor: Ing. Hector Javier Paiz Ramos | **Fecha:** Octubre 2026 · San Miguel, El Salvador

Este manual proporciona las instrucciones técnicas verificadas para clonar, configurar y desplegar los cuatro módulos de la solución: la API REST transaccional (Laravel 11), el punto de venta web (Vue 3 / Vite), el Data Warehouse analítico (PostgreSQL 16 en Docker), el pipeline de carga (Python ETL) y los tableros analíticos (Streamlit).

1. Arquitectura de Servicios y Mapa de Puertos

Antes de iniciar los servicios, valide que los puertos de red locales se encuentren libres en el sistema anfitrión:

Componente / Capa	Directorio	Tecnología Base	Puerto / URL de Acceso
Data Warehouse (OLAP)	`DataWareHouse/`	PostgreSQL 16 (Docker)	`localhost:5433` (`inventory_dw`)
API Transaccional (OLTP)	`InventaryAppApi/`	PHP 8.2+ / Laravel 11	`http://127.0.0.1:8000`
Punto de Venta POS	`InventaryApp/`	Vue.js 3 / Vite 5	`http://localhost:5173`
Pipeline ETL y Validación	`ProcesoETL/`	Python 3.10+ / pytest	Ejecución CLI por lotes
Plataforma BI Analítica	`Dashboards/`	Streamlit 1.36+ / Plotly	`http://localhost:8501`

2. Requisitos Previos del Sistema

El entorno de ejecución requiere tener instaladas las siguientes herramientas en el sistema operativo:

- **Git:** Control de versiones para la descarga y actualización del repositorio.
- **Docker Desktop:** Motor de contenedores activo (Linux containers) para la base dimensional.
- **PHP 8.2+:** Con extensiones habilitadas en `php.ini`: `pdo_pgsql`, `pgsql`, `mbstring`, `openssl`, `curl`, `fileinfo`.
- **Composer 2.x** (gestor PHP) y **Node.js 18+ / npm** (gestor y compilador de frontend).
- **Python 3.10+:** Con gestor `pip` y soporte para entornos virtuales (`venv`).

3. Procedimiento Paso a Paso de Instalación

Paso 0: Clonar el repositorio oficial

```
$ git clone https://github.com/cristianpined4/repo_proyecto_final_diplomado.git
$ cd repo_proyecto_final_diplomado
```

3.1 Despliegue del Data Warehouse Analítico (PostgreSQL en Docker)

El Data Warehouse debe iniciarse en primer lugar, pues alimenta tanto al pipeline ETL como a los tableros analíticos:

```
$ cd DataWareHouse
$ docker compose up -d
$ docker ps --filter "name=inventory_dw_pg"
# Credenciales: Host=localhost | Puerto=5433 | BD=inventory_dw | Usuario=dw_admin |
    Clave=dw_secure_password_2026
```

Inicialización Automatizada de Esquemas

El contenedor ejecuta los scripts DDL de la carpeta `sql/`, inicializando los esquemas `stg` (área intermedia de limpieza) y `dw` (esquema en estrella con tablas de dimensiones y hechos).

3.2 Configuración del Backend Operacional (Laravel 11 API)

Abra una nueva terminal e instale las dependencias y clave criptográfica de la API transaccional:

```
$ cd InventaryAppApi
$ composer install
$ cp .env.example .env
$ php artisan key:generate
```

Configure los parámetros de base de datos en `.env` (por defecto hacia Supabase o PostgreSQL local en puerto 5432):

```
DB_CONNECTION=pgsql
DB_HOST=aws-0-us-east-1.pooler.supabase.com
DB_PORT=5432
DB_DATABASE=postgres
DB_USERNAME=postgres.hnyatmpkozoebrcgjfrh
DB_PASSWORD=Inventario2026
```

Ejecute las migraciones y semillero de datos (o cargue `dataset_test.sql` desde la raíz para poblar 3 años históricos):

```
$ php artisan migrate:fresh --seed
$ php artisan serve
# La API queda escuchando solicitudes en http://127.0.0.1:8000
```

3.3 Despliegue del Frontend Web (Punto de Venta POS en Vue 3)

En una terminal independiente, descargue los módulos de Node.js e inicie el servidor de desarrollo Vite:

```
$ cd InventaryApp
$ npm install
$ npm run dev
# Acceda al punto de venta en el navegador: http://localhost:5173
```

3.4 Ejecución del Proceso ETL y Validación Automatizada

El pipeline en Python extrae los registros operacionales, aplica transformaciones en `stg` y consolida el histórico en `dw`:

```
$ cd ProcesoETL
$ python -m venv venv
$ venv\Scripts\activate          # En Linux/macOS: source venv/bin/activate
$ pip install -r requirements.txt
$ python main.py
```

Para comprobar la consistencia de los datos cargados y la lógica de transformación, ejecute la suite de pruebas unitarias:

```
$ pytest test_etl.py -v
# 5 pruebas automatizadas completadas exitosamente (PASSED en ~0.85s)
```

3.5 Despliegue de la Plataforma Analítica (Dashboards en Streamlit)

En otra terminal, inicie la aplicación interactiva de Business Intelligence con métricas de ventas y rotación:

```
$ cd Dashboards
$ pip install -r requirements.txt
$ python -m streamlit run app.py
# El tablero se abre de forma automatica en http://localhost:8501
```

4. Credenciales de Acceso Predeterminadas

A continuación se resumen las cuentas de usuario por defecto para autenticarse en los distintos componentes del sistema:

Módulo / Plataforma	Usuario	Contraseña	Rol y Permisos
Punto de Venta Web (POS)	`admin@plasticoslimeno.com`	`password`	Administrador General
Punto de Venta Web (Local)	`admin`	`admin`	Administrador de Turno
Data Warehouse (PostgreSQL)	`dw_admin`	`dw_secure_password_2026`	DBA Dimensional
Base Operacional (PostgreSQL)	`postgres.hnyatmpkozo...`	`Inventario2026`	Propietario OLTP

Seguridad en Entornos de Producción

Las credenciales antes listadas corresponden a entornos de evaluación académica y pruebas controladas. En un entorno productivo, estas claves deben sustituirse por valores protegidos y almacenados en un gestor de secretos seguro (Docker Secrets, AWS Secrets Manager o variables de entorno restringidas).

5. Diagnóstico y Resolución de Incidencias Comunes

Si experimenta inconvenientes durante el despliegue local, consulte la siguiente guía de diagnóstico rápido:

- **Puerto 5433 ocupado o en conflicto:** Si otra instancia local de PostgreSQL ya utiliza el puerto 5433, modifique el archivo `DataWareHouse/docker-compose.yml` cambiando el mapeo a `"5434:5432"`. Posteriormente, sincronice la variable `DW_PORT=5434` en los archivos `.env` de `ProcesoETL` y `Dashboards`.
- **Error `Driver [pgsql] not found` en Laravel:** La instalación de PHP en el sistema anfitrión no tiene activa la extensión nativa de PostgreSQL. Abra su archivo `php.ini`, descomente las siguientes directivas eliminando el punto y coma inicial:

  ```
  extension=pdo_pgsql
  extension=pgsql
  ```

  Guarde el archivo y reinicie su consola de comandos o servidor web.
- **Los tableros de Streamlit muestran filtros o métricas vacías:** Los tableros leen directamente las tablas de hechos `dw.fact_ventas` y `dw.fact_inventario_snapshot`. Este escenario ocurre cuando el Data Warehouse se encuentra recién creado pero aún no ha recibido la carga. Ejecute `python main.py` dentro del directorio `ProcesoETL` para cargar los 3 años de histórico.
- **Error de CORS en la comunicación Vue 3 ↔ Laravel API:** Verifique que el archivo `config/cors.php` del backend admita solicitudes desde el origen `http://localhost:5173` o contenga `'supports_credentials' => false` para rutas públicas de la API.